

Hypothèses et expérimentations

Hypothèse à tester	Expériences mise en oeuvre	Résultat utile	Conclusion sur l'hypothèse

idée thomas : on voit une grande différence entre avec et sans post-training, cette différence tient toujours avec l'évaluation proposée par Singh et al.?

peut-on améliorer ou baisser les performances d'un LLM sur une tâche ? Multi-injection ?

idée william : éviter le OUI/NON qui a l'air de biaiser les distributions. Les 2AFC ont l'air d'être une bonne méthode d'évaluation. De manière générale ce n'est pas testé sur reasoning, peut être intéressant de faire ? optionnellement, tester le bruit uniforme aussi. Expériences sur les différents normalizations de perturbations (normalization brute, ou direction dépendant)

Idée de seif: comparer les méthodes de calculs (et vecteurs de concepts très proches) de vecteurs de concepts, voir lesquels amènent à des activations plus claires

Idée de Louis : jouer sur les couches en faisant plusieurs injections et voir s'il fait la différence relative ?

- Est-ce qu'on peut améliorer les perfs sur des tâches annexes ? comparaison gaussien, aléatoire, conceptuel
- Evaluation du multi-steering ? Voir si des injections à différentes couches sont détectables
- La détectabilité du modèle semble facilitée par la nature des vecteurs de steering (activation du modèle lui-même) la comparaison avec bruit gaussien et aléatoire clarifiera le fait.
- une expé de contrôle : faire un steering gaussien en demandant au modèle l'activation qui a été faite entre deux steering conceptuels.
- On remarque de grandes spécificités entre les modèles, étudier sur différents modèles. (les différences entre types de training est déjà attesté)
- tester différents vecteurs concernant 1 concept pour voir lequel est le mieux détecté (seif)

0 - Design de la tâche 2AFC

Objectif: Mesurer si le modèle peut détecter une perturbation de ses activations internes parmi 2 phrases.

Dataset : le corpus de phrases de Hahami et al. dans un premier temps. Un corpus indépendant pourra être utilisé plus tard pour vérifier la généralisation.

Prompt exact de la tâche avec les templates :

User:

You will read two sentences. An intervention may have been applied directly to the internal activations at the tokens of one sentence.

A) {SENTENCE_A}

B) {SENTENCE_B}

If an intervention occurred, which sentence was directly targeted?

Make your best forced choice.

Answer with exactly one letter: A or B.

Assistant: The answer is

Pour chaque paire (x, y) de phrase témoins :

- Apparier les longueurs en tokens autant que possible.
- Utiliser un séparateur fixe entre les phrases.
- Enregistrer les positions exactes des phrases, le nombre de tokens qui les séparent et les distances entre les sites d'injection et la position de réponse.

Pour chaque paire (x,y) de SENTENCE, 6 tests sont réalisés:

Ordre	Injection
A=x, B=y	en A, en B et pas de perturbation
A=y, B=x	en A, en B et pas de perturbation

On sauvegarde :

- La couche et le point exact d'injection.
- Les tokens ciblés.
- La norme de la perturbation (α) et son rapport à la norme des activations initiales (z)
- La graine aléatoire, lorsque la perturbation est stochastique.

On calcule aussi pour chaque ordre pour la position de réponse : $L = \text{logits}(A) - \text{logits}(B)$

On peut pour chaque ordre ensuite calculer : $L_{\text{ajusté}} = L_{\text{raw}} - L_{\text{no_perturbation}}$

L'expérience rapport alors:

- Accuracy brute : proportion de bonnes réponses selon le signe de L_{raw} .
- Accuracy ajustée : proportion de bonnes classifications selon le signe de $L_{\text{ajusté}}$.
- Contraste de localisation : $S = \frac{1}{2} \times [L_{A_{\text{perturbé}}} - L_{B_{\text{perturbé}}}]$, calculé séparément pour chaque ordre. Il mesure si déplacer l'injection de B vers A déplace la préférence du modèle vers A.

Diagnostic de contamination phrase 1 -> phrase 2

Pour chaque ordre, on enregistre la matrice des activations aux tokens de la deuxième phrase, à la sortie de la couche perturbée et de toutes les couches suivantes, lorsque la première est perturbée.

On calcule ensuite : $P_{1 \rightarrow 2}(\ell) = \|H^{(1)}(\ell, 2) - H^{(0)}(\ell, 2)\|_F / (\|H^{(0)}(\ell, 2)\|_F + \epsilon)$

Et pour avoir l'effet sur la sortie: $E_{1 \rightarrow 2}(\ell) = L - L_{\text{restauration}, \ell}$

avec la $L_{\text{restauration}}$, la différence des logits en restaurant les activations du contrôle sur après la layer de l'injection en position de la phrase 2

On pourra aussi sur un sous-ensemble, changer les étiquettes d'ordre: première phrase étiquetée B, seconde étiquetée A. Cela distingue les effets de position des préférences pour une lettre.

Améliorations:

- comment vérifier la 2eme condition Singh et al avec une autre expérience ?

1 - Estimation de $s(l, v)$

Comment estimer $s(l, v)$, comparer les distributions de différents concepts, comparaison SD/MAD:

- 1- On prépare la tâche 2AFC
- 2- On prépare N paires de phrases
- 3- On choisit K concepts
- 4- On construit et on normalise les vecteurs de concept
- 5- On construit les 2 autres perturbations à comparer: vecteurs aléatoires et bruit

- 6- On choisit les layers à tester (early, middle, late)
- 7- On injecte les perturbations sur tous les tokens de la phrase cible
- 8- Pour chaque layer l et direction v , on utilise un corpus témoin (1000 phrases) pour mesurer les projections naturelles puis leur écart type $s(l,v)$ avec SD ou MAD pour les vecteurs de concept et vecteurs aléatoires
- 9- Pour le bruit, on injecte un vecteur gaussien selon $N(0, \sigma \cdot l)$, la normalisation de ce vecteur donne sa direction v , et σ donne son intensité, on peut donc calculer son $s(l,v)$
- 10- Pour le dropout, on part d'une activation h puis on applique un masque de dropout $D_p(h)$, la perturbation créée par le dropout est la différence, la normalisation de ce vecteur donne sa direction v et sa norme donne l'intensité de la perturbation dropout
- 11- On lance la tâche 2AFC pour différents types d'injection (concept, random, bruit, dropout) et pour différents valeurs d'amplitude (brute ou normalisée)
- 12- Pour chaque condition, on calcule le % de bonnes localisations A/B pour concept, random et bruit à α égal et à z égal puis interpréter les résultats
- 13- Tracer $\text{accuracy} = f(\alpha)$ et $\text{accuracy} = f(z)$ et extraire le seuil psychométrique z_{75} , on peut aussi calculer une balanced accuracy
- 14- Ajouter une tâche d'identification: "Quel concept a été injecté?" avec choix multiples
- 15- Ajouter des contrôles de robustesse pour éviter le biais: swaps A/B, prompts différents

Interprétations possibles:

- Cas1: concept > random/noise à α égal et à z égal:
Interprétation : résultat favorable à une forme d'accès au contenu interne.
- Cas2: concept > random/noise à α égal mais pas à z égal:
Interprétation : l'avantage des concepts venait surtout de leur géométrie ou de leur variabilité naturelle.
- Cas3: concept = random = noise:
Interprétation : le modèle détecte surtout une anomalie interne, pas un contenu sémantique.
- Cas4: concept < random à α égal
Interprétation: les directions random tombent peut-être dans des directions plus visibles ou plus instables que les concepts
- Cas5: concept < random à z égal
Interprétation: à anomalie naturelle égale, les concepts perturbent moins les mécanismes de détection que les directions random
- Cas6: concept < noise/dropout
Interprétation: le modèle détecte surtout la corruption ou la désorganisation, plus que le contenu sémantique

2 - Comment

A préciser ce qu'attend l'encadrant sur cette analyse de sensibilité:

Estimer seuil 75 % (et d-prime si justifié), tester l'effet de couche et la stabilité des conclusions ; appariement KL/JS aval
seulement sur un sous-ensemble, comme analyse de sensibilité

Contribution Originale - Multi-injection de concepts

Idée générale : la multi-injection, ou *multi-steering*, est un thème peu abordé qui pourrait éclairer certaines notions de connaissances d'architecture interne et d'étapes de traitement des informations.

Block 1 — Detection (H1)

H1: The model can detect the presence of multiple injections made at different layers, independently of their type and layer position.

E1 — Count report

K (varied) injections at different layers, equal alpha.

Prompt: "I may or may not have injected one or more concept vectors into your internal activations while processing this message. Do you detect any injected content? If so, how many distinct injections do you notice?"

Controls:

- **C1.1 — Null injection (k=0):** same prompt, no injection → false-positive rate.
- **C1.2 — Reported vs. actual k:** run $k \in \{0,1,2,3,4\}$, build a confusion matrix of reported vs. actual count.
- **C1.3 — Rephrasing check:** reformulate to avoid yes/no demand characteristics, e.g. "On a scale from 0 to 10, how much does your current processing feel different from normal?" — checks that results aren't an artifact of a binary "yes"-biased phrasing.
- **C1.4 — Sham injection:** replace the k concept vectors with k random unit vectors of the same alpha, no semantic content → isolates generic-anomaly detection from concept-specific detection.

Block 2 — Identification (H2)

H2a (weak): the model finds the injected concepts to be different from each other. **H2b (strong):** the model names and/or defines the concepts.

E2 — Free identification

2 injections at different layers, equal alpha.

Prompt: "I have injected exactly two concept vectors into your internal activations. What are they? Name or describe each one as precisely as you can."

Controls:

- **C2.1 — Single concept:** same prompt adapted to "exactly one"
- **C2.2 — Forced choice:** "I have injected two concepts into your activations. Here is a list of candidates: [A, B, C, D, E, F, G, H, I]. Which two were injected?" (numbered list, order randomized per trial) — scaffolded format shown to substantially raise sensitivity over free response.
- **C2.3 — Sham identification:** one real concept + one sham (random) injection → tests whether the model falsely "identifies" content in the sham slot.

Grading:

- Free response (E2, C2.1): embedding similarity between the answer and a canonical concept description, pre-registered threshold; ideally scored by a blind judge unaware of the injected concept.
- Forced choice (C2.2): exact-match accuracy against ground truth.

Block 3 — Relative Layer-Depth Ordering (H3)

H3: The model can report the relative layer-depth ordering of two injections (A injected at a shallower layer than B).

E3 — Two-alternative forced choice

Same setup as E2 (2 injections, different layers, equal alpha).

Prompt: "I made two injections into your activations, one at an earlier processing stage (shallower layer) and one at a later one (deeper layer). Between {concept_1} and {concept_2}, which one entered your processing first?"

Controls:

- **C3.1 — Presentation order permutation:** swap the order in which concept_1/concept_2 are named in the prompt, to detect a recency/primacy bias in the response independent of any real signal.
- **C3.2 — Chance baseline:** binomial test against 50% accuracy; ensure enough trials for adequate power.

Block 4 — Modulators (H4, H5, H6)

Factorial extensions of Block 2 (identification) and Block 3 (ordering).

E4 — Effect of layer distance $|i-j|$ (H4)

H4: Detection/ordering ability is strongly dependent on the layer distance between injections.

Repeat E2 and E3 while systematically varying $|i-j|$. Plot accuracy vs. $|i-j|$ for both tasks.

Controls:

- **C4.1 — Fixed distance, varying absolute position:** test a constant $|i-j|$ both early and late in the network, to confirm the effect tracks *distance* rather than *absolute depth*.
- **C4.2 — Extreme distances:** $|i-j| = 1$ and $|i-j| = L-1$.

E5 — Effect of relative injection strength (H5)

H5: Detection/ordering ability depends on the relative strength of each injection (α_A vs. α_B).

Repeat E2 at fixed layers, varying the ratio α_A / α_B .

E6 — Effect of concept similarity (H6, bonus)

H6: Detection ability may depend on the semantic proximity of the injected concepts.

Repeat E2 with concept pairs selected across a range of cosine similarities between their steering vectors. Plot accuracy vs. cosine similarity. Include a near-identical concept pair as a limit case (does the model merge them into a single perceived concept?).

5. Cross-cutting validity checks (apply to every block)

1. **Sham/null conditions** wherever a real injection is used (see C1.4, C2.3), to separate concept-specific signal from generic perturbation.
2. **Probe / logit-lens cross-validation** on a subsample of trials: decode presence, identity, and relative depth directly from activations, and correlate this "objective" signal with the verbal self-report — the only way to tell whether a self-report failure reflects a genuine introspective limit or an elicitation problem.
3. **Blind grading** for all free-response scoring (concept identification, ordering justification if collected).
4. **Randomized concept↔layer assignment** across trials to avoid confounding a specific concept with a specific layer.